

[Home](#) > [Interview](#) > 42 JavaScript Questions Towards Better Interv...

INTERVIEW

42 JavaScript Questions Towards Better Interviews

By Archana

Oct 02, 2024 | 8 Min Read | 5441 Views
(Last Updated)



‘JavaScript Interview questions’ – this phrase is one of the most searched Google queries. And Why?

JavaScript is the most popular programming language that every web browser understands! And therefore, it is the main ingredient on every web developers’ plate. Yes! The scripting language which was born with the then in-vogue programming language JAVA is now the core language that makes the web browsers all the more interactive.

Little wonder, that we meet here on this page to uncover the essential JavaScript Interview questions. Above all, a web developer



role is one of the most interesting and creative job roles that keep your pockets happy too!

Not only web development, many other jobs in Information Security Software Development, WordPress Development, etc. demand Javascript Developers.

So, not taking any more time of your's let's dive deep into the JavaScript questions!

Let's start with some overview questions. These are the brushing-up questions. Above all, you would encounter them in a more or less casual way. Everyone surely knows them but fails to put them into words. You need to know, how to answer them, that's all.



Table of contents

1. JAVASCRIPT OVERVIEW QUESTIONS

- What is JavaScript?
- 2. How to implement JavaScript?
- How do you write JavaScript?



- What are Objects in JavaScript?

- What are JavaScript Events?

2. Theoretical JavaScript Questions

- Can you name any two programming paradigms important for JavaScript app developers?

JAVASCRIPT OVERVIEW QUESTIONS

*Before diving into the next section, ensure you're solid on full-stack development essentials like front-end frameworks, back-end technologies, and database management. If you are looking for a detailed Full Stack Development career program, you can join GUVI's **Full Stack Development Course** with Placement Assistance. You will be able to master the MERN stack (MongoDB, Express.js, React, Node.js) and build real-life projects.*

*Additionally, If you want to explore JavaScript through a self-paced course, try GUVI's **JavaScript certification course**.*

1. What is JavaScript?

Ans. JavaScript is the world's most popular programming language. Above all, it is the programming language mainly used in the Web Development domain. and is commonly implemented for creating web pages.

JavaScript is a

- high-level,
- dynamic,
- object-based, and
- interpreted programming language.

JavaScript was invented by Brendan Eich in 1995. It was developed for Netscape 2 and became the ECMA-262 standard in 1997.



It is commonly used for creating web pages. JavaScript allows us to add dynamic behavior to the webpage and add special effects to it. On websites, JavaScript is mainly used for validation purposes.

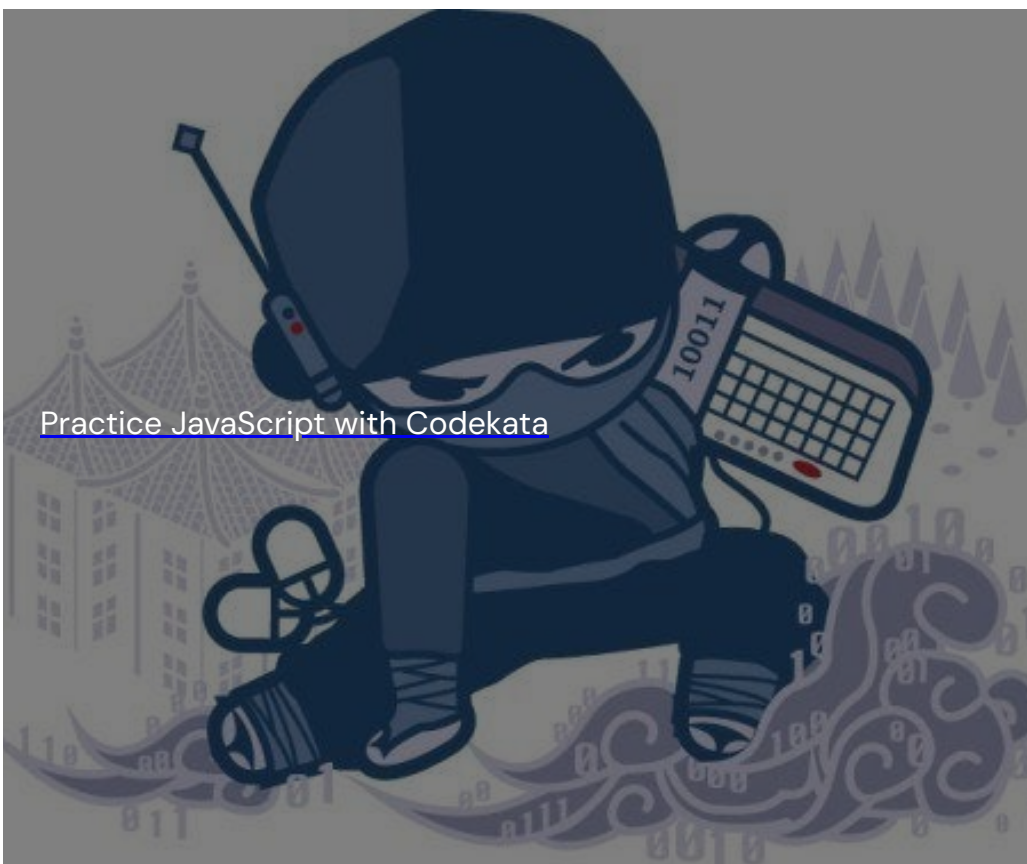
It is widely used, supported & accessible even to beginners.

2. How to implement JavaScript?

Ans. JavaScript can be implemented in three different ways namely

- Embedding code
- Inline code
- External file

3. How do you write JavaScript?



[Practice JavaScript with Codekata](#)

Ans. JavaScript code is embedded in the HTML. The client browsers can then interpret and run them.



- JavaScript code is case-sensitive.
- You can ignore the white space between words and tabs.
- Also, you can ignore the line breaks except within a statement.
- JavaScript statements end with a semi-colon ‘;’.

**Elevate Your Career Zen Class
Courses with Placement Guidance**

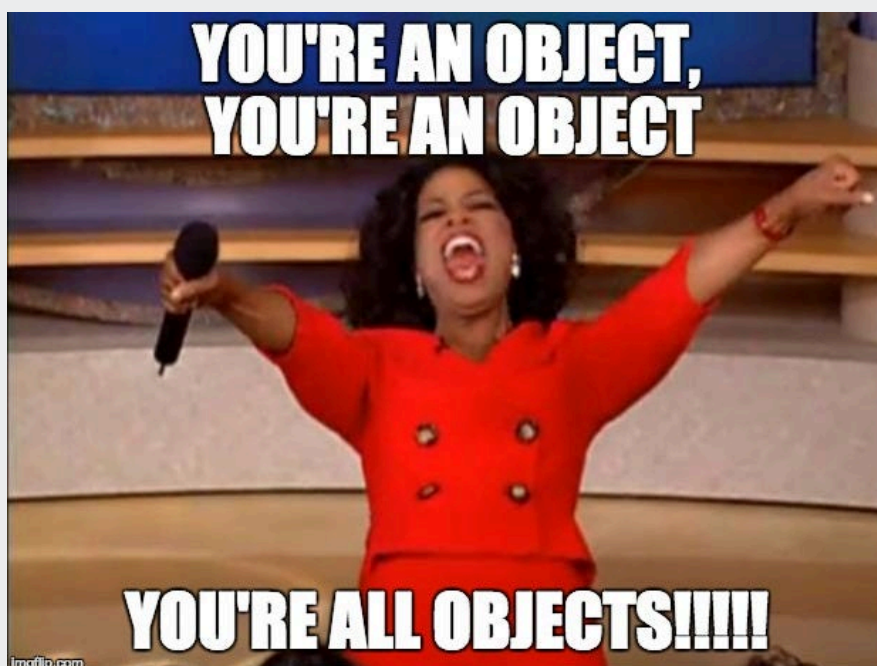
[Learn More](#)



4. What are Objects in JavaScript?

Ans. In JavaScript, an object is a standalone entity, with properties and type. Compare it with a cup,

for example. A cup is an object, with properties. A cup has a color, a design, weight, etc.



JavaScript properties: The same way, JavaScript objects can have properties, which define their characteristics

5. What are JavaScript Events?

Ans. The change in the state of an object is known as an Event. In HTML, there are various events that represent that some activity is performed by the user or by the browser.

When JavaScript code is included in HTML, JavaScript reacts over these events and allows the execution. This process of reacting over the events is called Event Handling. Thus, JS handles the HTML events via Event Handlers.

Common events are:

- **onLoad** – occurs when a page loads in a browser
- **onUnload** – occurs just before the user exits a page
- **onMouseOver** – occurs when you point to an object
- **onMouseOut** – occurs when you point away from an object
- **onSubmit** – occurs when you submit a form
- **onClick** – occurs when an object is clicked

JavaScript's interaction with HTML is handled through events that occur when the user or the browser manipulates a page. When the page loads, it is called an event. When the user clicks a button, that clicks to is an event.

Other examples include events like pressing any key, closing a window, resizing a window, etc.

Developers can use these events to execute JavaScript coded responses, which cause buttons to close windows,

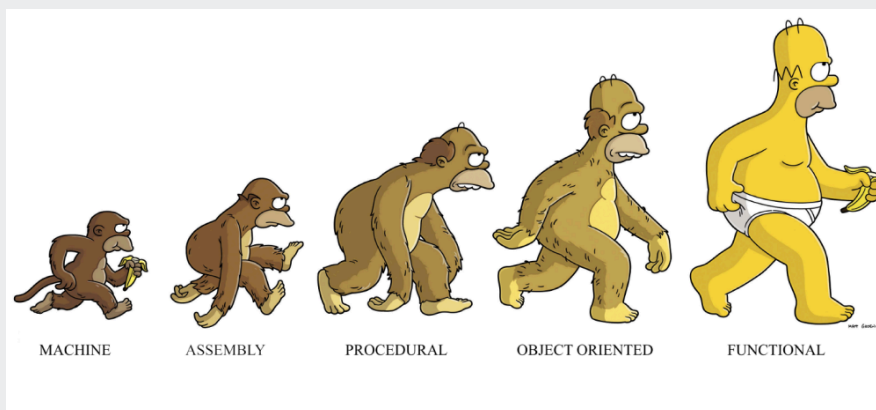


messages to be displayed to users, data to be validated, and virtually any other type of response imaginable. Events are a part of the Document Object Model (DOM) Level 3 and every HTML element contains a set of events that can trigger JavaScript Code.

Let's move ahead. With more interesting JavaScript Questions.

Theoretical JavaScript Questions

6. Can you name any two programming paradigms important for JavaScript app developers?



JavaScript is a multi-paradigm language, supporting imperative/procedural programming along with OOP (Object-Oriented Programming) and functional programming. JavaScript supports OOP with prototypal inheritance.

7. What is functional programming?

Functional programming produces programs by composing mathematical functions. Furthermore, it avoids shared state & mutable data.

Lisp (specified in 1958) was among the first languages to support functional programming and was heavily inspired by lambda calculus. Functional programming is an essential concept in JavaScript (one of the two pillars of JavaScript).

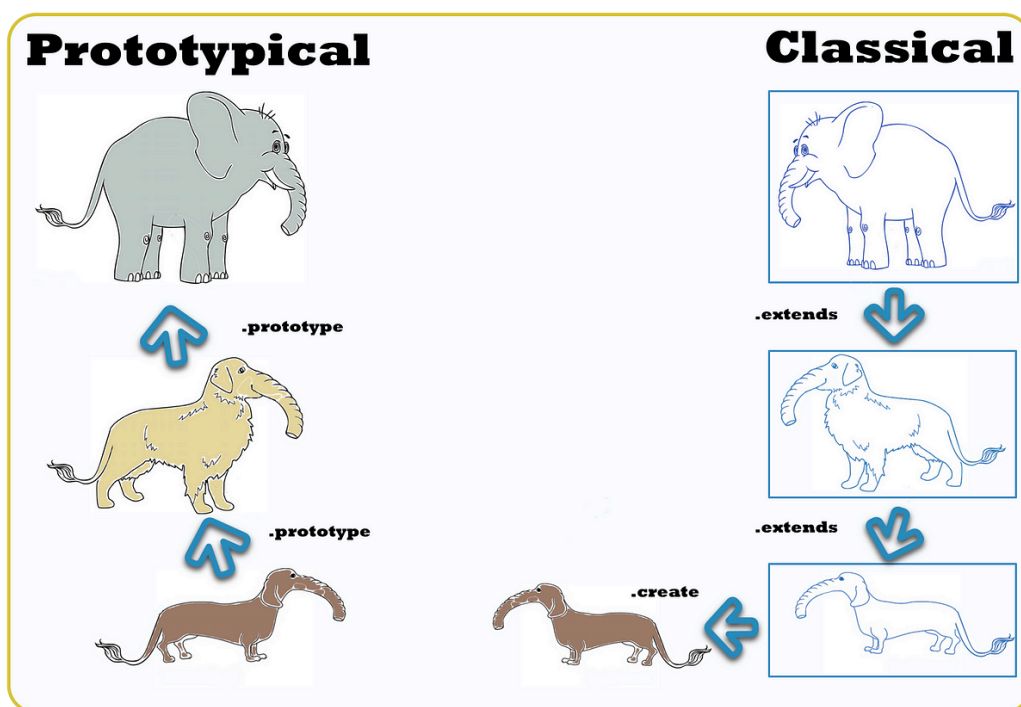


Several common functional utilities were added to JavaScript in ES5.

The core focus lies on:

- Function purity
- Avoiding side-effects
- Simple function composition

8. What is the difference between classical inheritance and prototypal inheritance?



Taken from Medium

Class Inheritance: Instances inherit from classes (like a blueprint – a description of the class), and create sub-class relationships: hierarchical class taxonomies. Instances are typically instantiated via constructor functions with the `new` keyword. In short, Class inheritance may or may not use the `class` keyword from ES6.

Prototypal Inheritance: Instances inherit directly from other objects. Instances are typically instantiated via factory functions or `Object.create()`. Instances may be



composed of many different objects, allowing for easy selective inheritance.

9. When is prototypal inheritance an appropriate choice?

There is more than one type of prototypal inheritance:

- Delegation (i.e., the prototype chain).
- Concatenative (i.e. mixins, `Object.assign()`).
- Functional (Not to be confused with functional programming. A function used to create a closure for private state/encapsulation).

Each type of prototypal inheritance has its own set of use-cases. However, all of them are equally useful in their ability to enable composition. That will create has-a or uses-a or can-do relationships as opposed to the is-a relationship created with class inheritance.

10. What is asynchronous programming, and why is it important in JavaScript?



Synchronous programming means that, barring conditionals and function calls, code is executed sequentially from top-to-bottom, blocking on long-running tasks such as network requests and disk I/O.

Whereas, **Asynchronous programming** means that the engine runs in an event loop. When a blocking operation is needed, the request is started, and the code keeps running without blocking for the result.

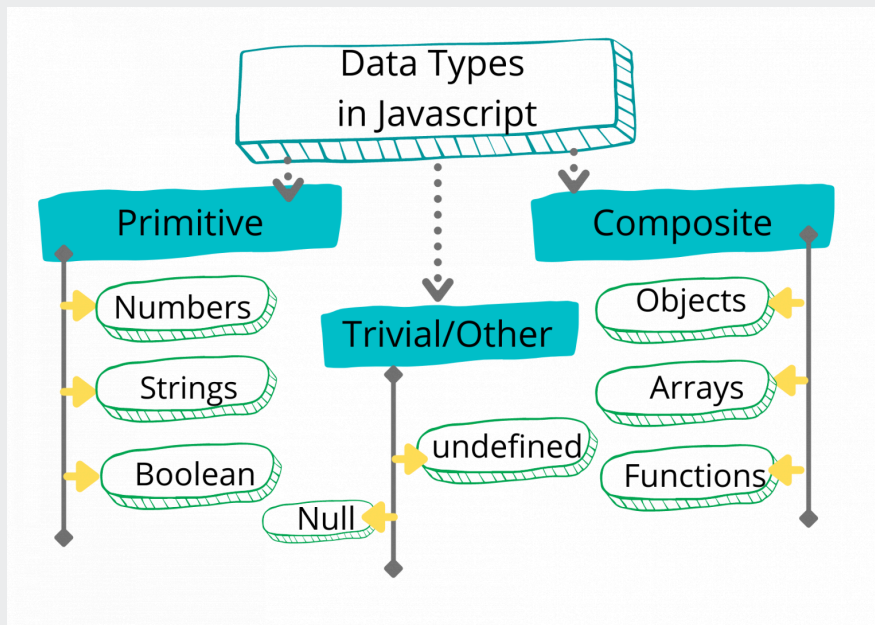
When the response is ready, an interrupt is fired, which causes an event handler to be run, where the control flow continues. In this way, a single program thread can handle many concurrent operations.

User interfaces are asynchronous by nature and spend most of their time waiting for user input to interrupt the event loop and trigger event handlers. Node is asynchronous by default. That means the server works in much the same way, waiting in a loop for a network request. And accepting more incoming requests while the first one is being handled.

This is important in JavaScript, because it is a very natural fit for user interface code, and very beneficial to performance on the server.

11. What are JavaScript data types?





In JavaScript, there are

3 Primary Data Types:

String,

Number,

Boolean.

Then, 4 Composite Data Types

Object

Functions

Date,

Array.

And, two data types that cannot contain any values:

Null and

Undefined

12. What is the difference between "==" and "==="?

"==" checks only for equality in value whereas "===" is a stricter equality test and returns false if either the value or



the type of the two variables are different.

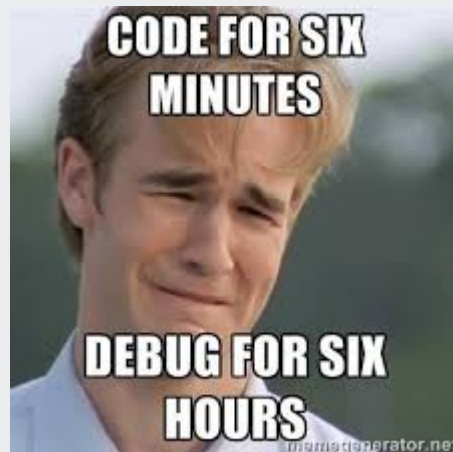
So, the second option needs both the value and the type to be the same for the operands.

13. What is an undefined value in JavaScript?

Undefined value means the:

- Variable used in the code doesn't exist
- Furthermore, the variable is not assigned to any value
- Property doesn't exist

14. What are the different types of errors in JavaScript?



There are three types of errors:

- **Load time errors:**

Errors that come up when loading a web page like improper syntax errors are known as Load time errors and it generates the errors dynamically.

- **Run time errors:**

Errors that come due to misuse of the command inside the HTML language.

- **Logical Errors:**



These are errors that occur due to the wrong logic performed on a function.

15. Define event bubbling

JavaScript allows DOM elements to be nested inside each other. In such a case, if the handler of the child is clicked, the handler of the parent will also work as if it were clicked too.

Further ahead, moving onto more general JavaScript Questions. These are very interesting at the same time very likely asked JavaScript Interview Questions.

General Questions

16. What is the 'this' keyword in JavaScript?

In JavaScript, this keyword refers to the object it belongs to.

this has different values depending on where it is used:

=> In a method, this refers to the owner object.

=> Alone, this refers to the global object.

=> In a function, in strict mode, this is undefined

=> While, in a function, this refers to the global object.

=> In an event, this keyword refers to the element that received the event.

=> Methods like call() and apply() will assign this to any object.

=> In an object, this refers to the "owner" of the method.



17. What is the difference between ViewState and SessionState?

ViewState is specific to a page in a session.

SessionState is specific to user-specific data that can be accessed across all pages in the web application.

18. Does JavaScript support automatic type conversion?

Yes, JavaScript does support automatic type conversion. It is the common way of type conversion used by JavaScript developers.

19. How can you read and write a file using JavaScript?

There are two ways to read and write a file using JavaScript

- Using JavaScript extensions
- Using a web page and Active X objects

20. How can you read and write a file using JavaScript?

The `parseInt()` function is used to convert numbers between different bases. `parseInt()` takes the string to be converted as its first parameter. And the second parameter is the base of the given string.





Stress Interview Questions

21. What is the use of Void(0)?

Void(0) is used to prevent the page from refreshing. We pass the parameter "zero" when calling. Void(0) is used to call another method without refreshing the page.

22. What are JavaScript Cookies?

A cookie is **an amount of information that persists** between a server-side and a client-side. A web browser stores this information at the time of browsing.

A cookie contains the information as a string generally in the form of a name-value pair separated by semi-colons.

23. How to target a particular frame, from a hyperlink, in JavaScript?

This can be done by including the name of the required frame in the hyperlink using the 'target' attribute.

There are two approaches mainly followed:

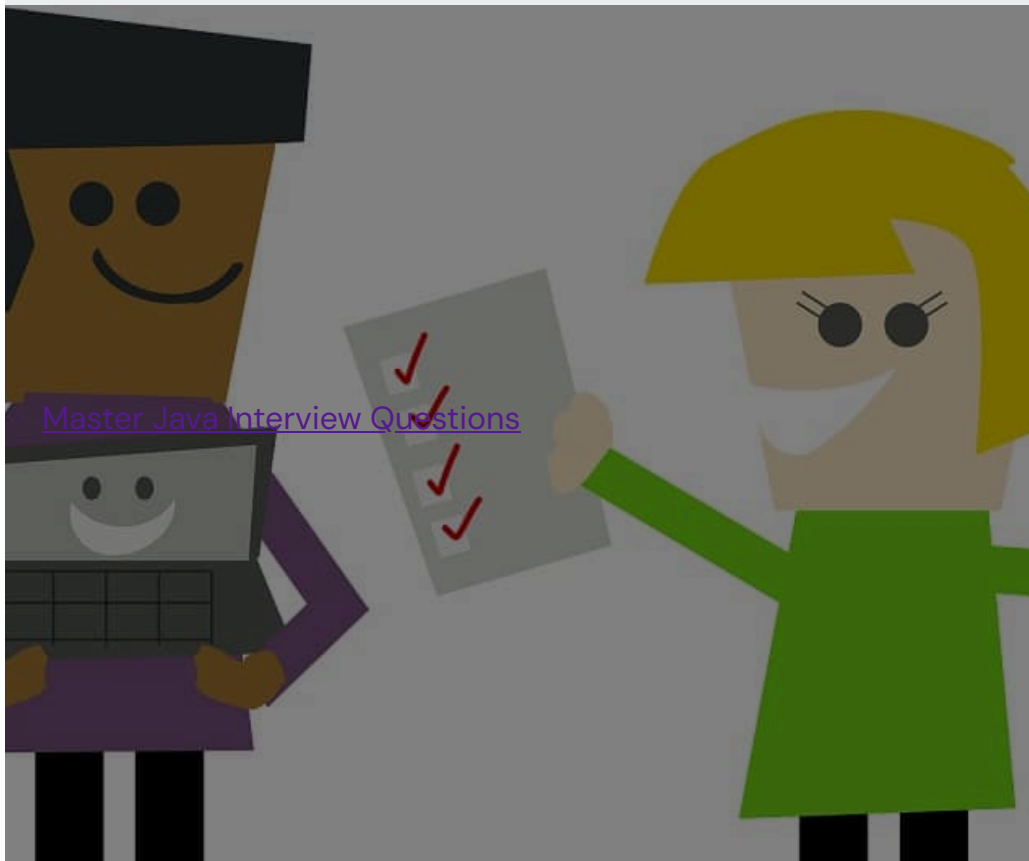


Target frame without using JavaScript. // Target attribute has iframe name as its value.

```
< a href = "#" target = "frame-name" >Link</ a > //Name attribute has name of the iframe.
```

And the other is Target frame using JavaScript: //General syntax.

```
document.getElementById( "frame-id" ).src = "url";
```



24. Explain the role of deferred scripts in JavaScript?

By default, the parsing of the HTML code, during page loading, is paused until the script has not stopped executing. That means, if the server is slow or the script is particularly heavy, then the webpage is displayed with a delay.

While using Deferred, scripts delays execution of the script till the time HTML parser is running. This reduces the loading time of web pages and they get displayed faster.



25. What does a timer do and how would you implement one?

Setting timers allows you to execute your code at predefined times or intervals.

This can be achieved through two main methods:

setInterval(); and

setTimeout();

setInterval() accepts a function and a specified number of milliseconds. ex) setInterval(function(){alert("Hello, World!")},10000) will alert the "Hello, World!" function every 10 seconds.

Whereas, setTimeout() also accepts a function, followed by milliseconds. setTimeout() will only execute the function once after the specified amount of time, and will not reoccur in intervals.

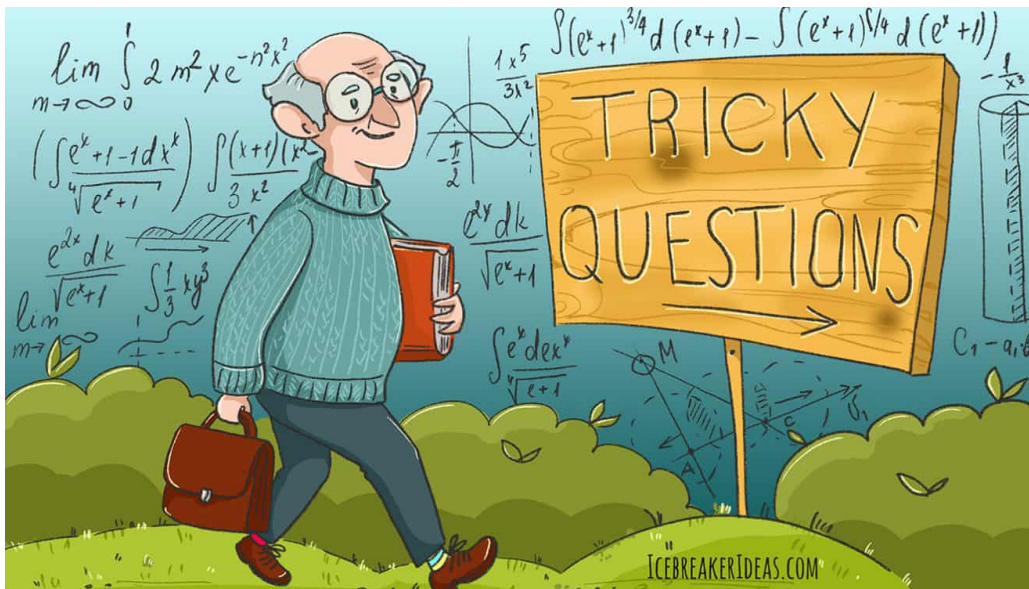
26. Explain the concept of unobtrusive Javascript?

Unobtrusive JavaScript is basically a JavaScript methodology that seeks to overcome browser inconsistencies by separating page functionality from structure.

The basic premise of unobtrusive JavaScript is that page functionality should be maintained even when JavaScript is unavailable on a user's browser.

Some Tricky JavaScript Questions





27. Is false false?

No. Because it's a string with a length greater than 0. Only empty strings can be considered false.

28. Is ' ' false?

No.

Because it's not an empty string. There is a white space in it.

29. Is Boolean(function(){}) true or false?

true

If you pass a true value to Boolean, it will be true.

30. What is 2 + true?



3

The plus operator between a number and a boolean or two booleans will convert a boolean to a number. Hence, true converts to 1, and you get the result of 2+1.

31. What is 6+9?

69

If one of the operands of the plus (+) operator is a string it will convert the other number or boolean to string and perform concatenation. For the same reason, "2"+true will return "2true".

32. What is the value of +dude?

NaN.

The plus (+) operator in front of a string is a unary operator that will try to convert the string to a number. Here, JavaScript will fail to convert the "dude" to a number and will produce NaN.

33. If you have var y = 1, x = y = typeof x; What is the value of x?

"undefined".



34. If `var a = (2, 3, 5)`; what is the value of `a`?

5

The comma operator evaluates each of its operands (from left to right) and returns the value of the last operand.

35. What is `-5%2`?

-1

The result of remainder always gets the symbol of the first operand.

Code Output Questions



36. What will the code below output to the console and why?




```
(function(){  
  var a = b = 3;  
})();  
console.log("a defined? " + (typeof a !==  
'undefined'));  
console.log("b defined? " + (typeof b !==  
'undefined'));
```

Answer:

Since both a and b are defined within the enclosing scope of the function. And since the line, they are on begins with the var keyword, most JavaScript developers would expect typeof a and typeof b to both be undefined in the above example.

However, that is not the case. The issue here is that most developers incorrectly understand the statement var a = b = 3; to be shorthand for:

```
var b =3; var a =b;
```

But in fact, var a =b =3; is actually shorthand for:

```
b =3; var a =b;
```

As a result (if you are not using strict mode), the output of the code snippet would be:

```
a defined? false
```

```
b defined? true
```

37. What will the code below output to the console and why?

```
var myObject = {  
  foo: "bar",  
  func: function() {  
    var self = this;
```



```
console.log("outer func: this.foo = " + this.foo);
console.log("outer func: self.foo = " + self.foo);
(function() {
  console.log("inner func: this.foo = " + this.foo);
  console.log("inner func: self.foo = " + self.foo);
})();
JavaScript Interview Questions 10 / 28
}
};
myObject.func();
```

Answer:

The above code will output the following to the console:

outer func: this.foo = bar

outer func: self.foo = bar

inner func: this.foo = undefined

inner func: self.foo = bar

In the outer function, both `this` and `self` refer to `myObject` and therefore both can properly reference and access `foo`.

Again, in the inner function, though, `this` no longer refers to `myObject`. As a result, `this.foo` is undefined in the inner function, whereas the reference to the local variable `self` remains in scope and is accessible there.

(Prior to ECMA 5, `this` in the inner function would refer to the global window object; whereas, as of ECMA 5, `this` in the inner function would be undefined.)

38. What will the code below output? Explain your answer.

```
console.log(0.1 + 0.2);
console.log(0.1 + 0.2 == 0.3);
```



Answer:

An educated answer to this question would simply be: "You can't be sure. it might print out "0.3" and "true", or it might not.

Numbers in JavaScript are all treated with floating point precision, and as such, may not always yield the expected results."

The example provided above is a classic case that demonstrates this issue. Surprisingly, the output here is:

0.30000000000000004

false

39. What will be the output when the following code is executed? Explain.

```
console.log(false == '0')  
console.log(false === '0')
```

Answer:

Firstly, the code will output:

true

false

In JavaScript, there are two sets of equality operators.

Here, the triple-equal operator === behaves like any traditional equality operator would. So, it evaluates to true if the two expressions on either of its sides have the same type and the same value.

The double-equal operator, however, tries to coerce the values before comparing them. It is therefore generally good practice to use the === rather than ==. The same holds true for !== vs !=.



40. What will the code below output to the console and why?

```
var arr1 = "john".split("");
var arr2 = arr1.reverse();
var arr3 = "jones".split("");
arr2.push(arr3);
console.log("array 1: length=" + arr1.length + "
last=" + arr1.slice(-1));
console.log("array 2: length=" + arr2.length + "
last=" + arr2.slice(-1));
```

Answer:

The logged output will be:

"array 1: length=5 last=j,o,n,e,s"

"array 2: length=5 last=j,o,n,e,s"

arr1 and arr2 are the same after the above code is executed for the following reasons:

Calling an array object's `reverse()` method doesn't only return the array in reverse order, it also reverses the order of the array itself (i.e., in this case, arr1).

The `reverse()` method returns a reference to the array itself (i.e., in this case, arr1). As a result, arr2 is simply a reference to (rather than a copy of) arr1. Therefore, when anything is done to arr2 (i.e., when we invoke `arr2.push(arr3);`), arr1 will be affected as well since arr1 and arr2 are simply references to the same object.

CODE In JavaScript



HAPPINESS IS



**...when your code
runs without error.**

41. Implement *moveLeft* animation

Answer:

Firstly, use `setInterval` that will place the element to the left position by some pixels every 10ms.

Then, you will see the element moving towards the desired position. When you call `setInterval`, it returns a `timeld`. After reaching the respective location, you have to clear the time interval. So that function will not be called again and again every 10ms.

```
function moveLeft(elem, distance) {  
  var left = 0;  
  function frame() {  
    left++;  
    elem.style.left = left + 'px';  
    if (left == distance)  
      clearInterval(timeld)  
  }  
  var timeld = setInterval(frame, 10); // draw every 10ms  
}
```



42. Implement cache to a recursive *fibonacci* function

Answer: If fibonacci is already calculated it is served from the global memo array otherwise it is calculated.

```
var memo = [];  
  
function _fibonacci(n) {  
  
  if(memo[n]) {  
  
    return memo[n];  
  
  }  
  
  else if (n < 2) {  
  
    return 1;  
  
  }  
  
  else {  
  
    fibonacci(n-2) + fibonacci(n-1);  
  
  }  
  
}
```

**Elevate Your Career Zen Class
Courses with Placement Guidance**

[Learn More](#)



Wrapping Up

So, these are 42 questions towards better JavaScript Programming.

To clarify, the questions are taken from the book JavaScript Interview Questions & Answers, Boosting your web development Career, Fabio CIMO. Found it very helpful so sharing it across.

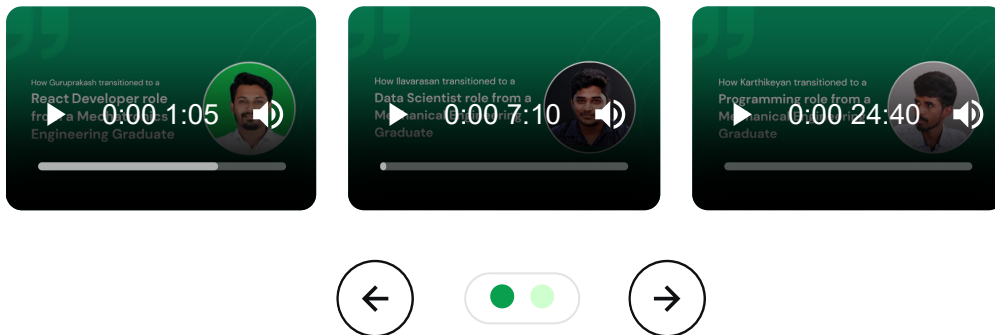
Kickstart your Full Stack Development journey by enrolling in GUVI's certified **[Full Stack Development Course](#)** with Placement Assistance where you will master the MERN stack (MongoDB,



Express.js, React, Node.js) and build interesting real-life projects. This program is crafted by our team of experts to help you upskill and assist you in placements.

If you want to explore JavaScript through a self-paced course, try GUVI's [JavaScript certification course](#).

Career transition





About the Author

Archana

A traveler, and explorer, Archana is an active writer at GUVI. You can usually find her with a book/kindle binge-reading (with an affinity to the mystery/humor).

Connect with me @



Did you enjoy this article?

Name

Email

Rate this Article ☆ ☆ ☆ ☆ ☆

Enter Your Comment



Submit My Comment



Learn

Full Stack Development

in your favourite programming stack

- MERN, MEAN, Java, Python & more
- Mentors from Global Companies
- Build Portfolio with 20+ Projects

Enroll Now

**Learn in english, தமிழ், ಕನ್ನಡ & हिंदी*





SOUTH ASIAN
WOMEN IN TECH

Join the World's Largest Women-Only Gen AI Learning Challenge



Sept 21st, 2024

Exclusive Access & Celebrations

Learnathon & Hackathon

Courses & Certifications

Register Now



Get In Touch For Details!

Name*

Email ID*



Phone Number*



Select your interested program*

Choose Program



Get In Touch Request Information

Recommended Courses

Career Programs

Micro Courses





Java Full Stack Development Course

Available in

ENGLISH TAMIL

[Know More >>](#)

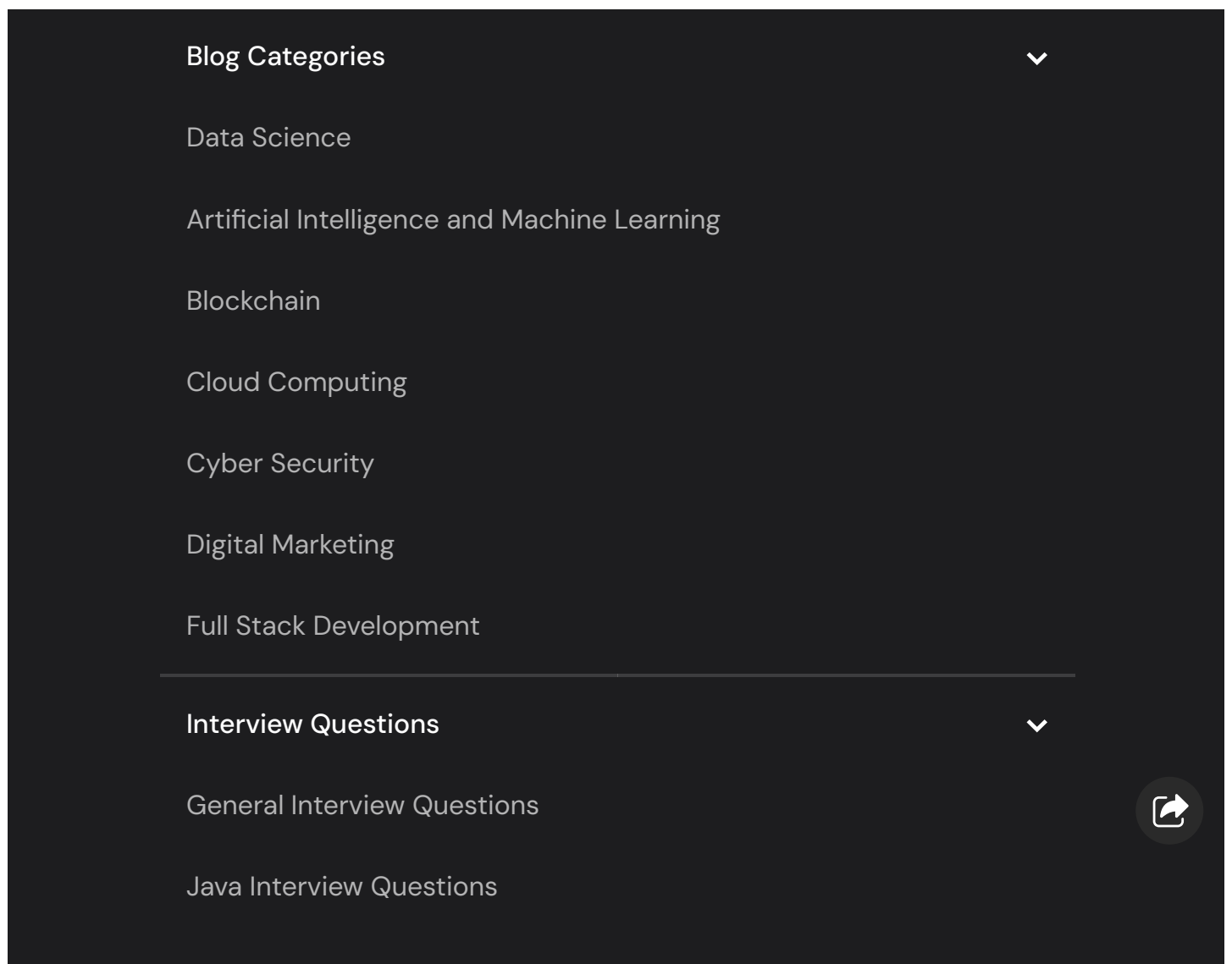
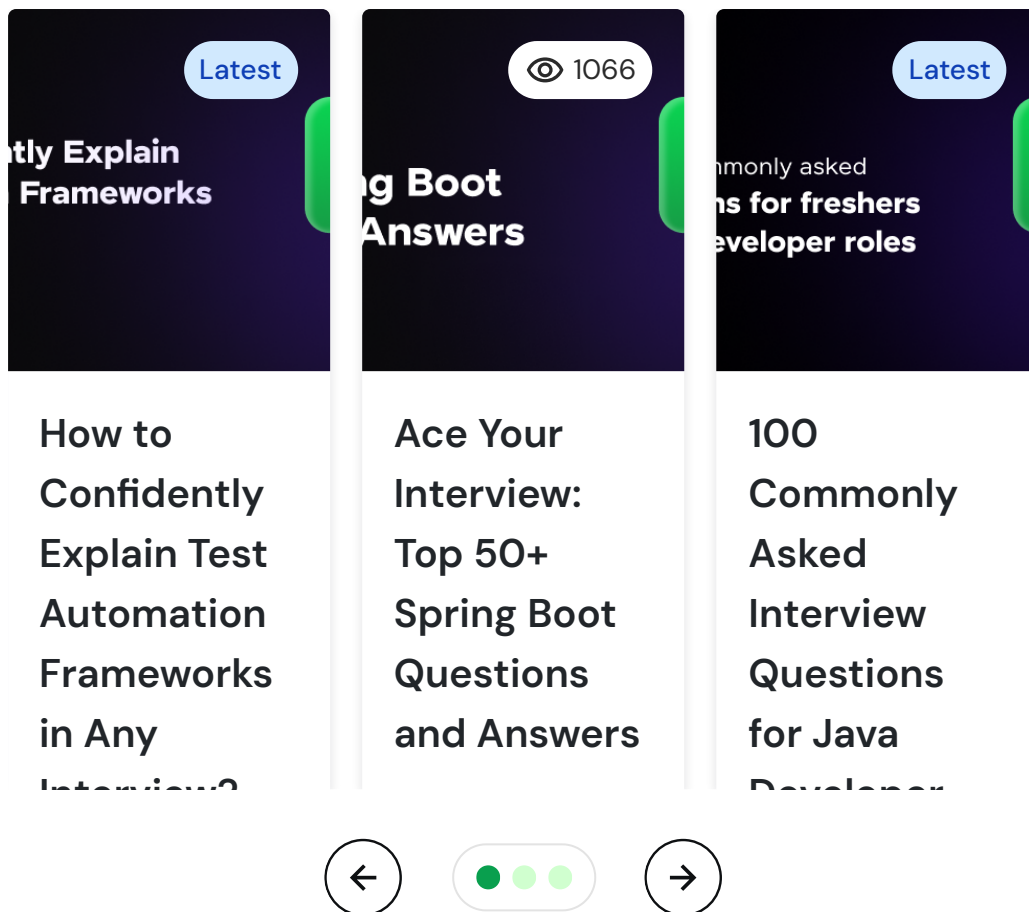


[Schedule 1:1 free counselling](#)

[Talk to Career Expert](#)

Similar Articles





[Python Interview Questions](#)

[SQL Interview Questions](#)

[Selenium Interview Questions](#)

[Data Science Interview Questions](#)

Salary blog



[UI/UX Designer Salary](#)

[Data Scientist Salary](#)

[Full-Stack Developer Salary](#)

[Motion Graphics Designer Salary](#)

[Cloud Computing Engineer Salary](#)

[Digital Marketer Salary](#)

About us



[Our Story](#)

[Careers](#)

[Refund Policy](#)

[FAQs](#)

[Contact Us](#)



[Refer & Earn](#)

GUVI – India's Pioneering Vernacular EdTech, incubated by IIT-M, IIM-A, and a prestigious part of the HCL group. Empowering over 2.5 million global learners with top-tier educational solutions through a vernacular approach, we have revolutionized global tech education with strategic partnerships, including 'Google for Education,' AICTE,



AWS, Google Cloud, IIT-M Pravartak, UiPath, and NASSCOM. GUVI is your trusted guide to inclusive and accredited learning experiences.



Follow us on



[Terms and Conditions](#)

[Privacy Policy](#)

© GUVI Geeks Network Pvt. Ltd

 [Table of contents](#)

